Computer Vision Engineer – Autonomous Systems

Technical Assessment · Anarampower Ltd – (Skyscouter) · Port Coquitlam, BC

Duration	5 days from receipt of this package
Submission	GitHub / GitLab repo link + screen-recorded demo video + executable run.sh
Language	Python 3.10+ required. C++ extensions optional.
Hardware	CPU or GPU both permitted. If GPU used, document exact model, VRAM, and CUDA version in README.
Assets provided	input.mp4 (test video), calib.json (camera calibration), track_bin.py (skeleton), run.sh (template)
Contact	sarabi@gmail.com — questions within first 6 hours only

Scenario

A fixed monocular camera is mounted on a static pole at height 1.35 m, pitched 15° downward. A standard outdoor garbage bin (approximately 0.40 m diameter × 0.65 m tall) is placed on the ground at distances of 5–9 m from the pole base. During the sequence the bin is moved to three positions. A person walks briefly through the scene at each position, causing partial occlusion.

Your system must: (1) detect the garbage bin in real time, (2) estimate its 3D position in the camera frame and transform it to a world frame fixed at the pole base, and (3) output a live coordinate stream to stdout as frames are processed. Three ground-truth positions are marked on the floor with coloured tape for validation.

1 Detection — 30 pts

required — 1a. Garbage bin detection pipeline — 15 pts

Using the provided video clip, detect the garbage bin in every frame. You may use any detector (YOLOv8, Faster-RCNN, custom, or classical). For each frame output a bounding box [x1, y1, x2, y2] and a confidence score. Note: standard COCO-pretrained models include a 'trash can' / 'garbage bin' class — document whether you use it directly or fine-tune, and why.

- Detection rate > 90% across the full sequence
- Inference < 100 ms/frame on GPU, < 250 ms/frame on CPU
- Bounding box IoU > 0.6 against provided ground-truth boxes (supplied after submission)

required — 1b. Occlusion continuity — 15 pts

The sequence includes frames where a person briefly walks in front of the bin. You must maintain detection continuity — do not simply drop or skip those frames. Describe your strategy explicitly in the README.

bonus **1c. Model choice justification** +10 pts bonus

If you fine-tune a model or construct a custom dataset, document it quantitatively (precision/recall delta or mAP gain). A generic 'I used YOLOv8' with no justification scores zero on this sub-task.

2 3D Localisation 40 pts

required **2a. Distance estimation from bounding box** 20 pts

The bin dimensions are known (0.40 m diameter × 0.65 m tall). Using the provided camera intrinsics, estimate the distance from the camera origin to the bin centroid in each frame. Acceptable error: ±0.30 m at 7 m range.

You must show your geometric derivation in your README or as inline comments. Code-only submissions score partial credit only.

Calibration file format:

```
// calib.json
{
  "K": [[fx, 0, cx], [0, fy, cy], [0, 0, 1]],
  "dist_coeffs": [k1, k2, p1, p2, k3],
  "camera_height_m": 1.35,
  "camera_tilt_deg": -15.0,
  "image_width_px": 1920,
  "image_height_px": 1080
}
```

required **2b. 3D position in camera frame** 10 pts

Output the (X, Y, Z) position of the bin centroid in the camera coordinate frame for each frame (Z = depth along optical axis). Format: timestamped CSV.

```
frame_id, timestamp_ms, x_cam, y_cam, z_cam, confidence
0042,     1400,         -0.23,  0.61,  6.87,  0.94
```

required **2c. Transform to world frame** 10 pts

Define a world frame with origin at the base of the camera pole. Apply the camera extrinsics (height + tilt from calib.json) to transform camera-frame coordinates to world-frame coordinates, where z_world = height above ground.

```
frame_id, t_ms, x_cam, y_cam, z_cam, x_world, y_world, z_world, conf
0042,     1400, -0.23,  0.61,  6.87,   6.71,    -0.21,    0.10,  0.94
```

bonus **2d. Error analysis vs. ground-truth waypoints** +10 pts bonus

The video includes 3 floor waypoints (coloured tape, known to ±2 cm). Compute your system's localisation RMSE against these reference positions and produce a plot of error vs. distance (error_vs_distance.png).

3 Real-time Tracking Output 30 pts

required **3a. Live coordinate stream** 15 pts

Implement a script that processes the video frame-by-frame and prints world-frame coordinates in real time to stdout. The first output line must appear within 2 seconds of launch. Do not buffer the entire video.

```
$ bash run.sh --video input.mp4 --calib calib.json
[frame 001] bin @ world (6.82, -0.18, 0.09) m  conf=0.97  dt=34ms
[frame 002] bin @ world (6.81, -0.19, 0.09) m  conf=0.96  dt=31ms
[frame 047] bin @ world (4.12,  1.44, 0.11) m  conf=0.91  dt=38ms
[frame 048] OCCLUDED — last known (4.12, 1.44, 0.11) m  age=2fr
```

required **3b. Trajectory visualisation** 15 pts

Generate a top-down 2D plot (matplotlib or OpenCV overlay) showing the bin trajectory in the world XY plane across the full sequence. Mark the 3 known stop positions distinctly. Save as trajectory.png in the repo root.

bonus **3c. Kalman filter smoothing** +10 pts bonus

Wrap your position estimates in a Kalman filter (constant-velocity model). Show raw vs. filtered position plots. Quantify jitter reduction (std dev of stationary readings). Calling a library without explaining the state vector scores partial credit only.

bonus **3d. Edge deployment notes (Jetson Orin NX)** +10 pts bonus

In a dedicated README section, describe how you would adapt this pipeline to run on a Jetson Orin NX companion computer aboard a moving UAV. Cover:

- Model quantisation strategy (INT8 / FP16, TensorRT vs. RKNN)
- How the coordinate transform changes when the camera is no longer fixed (IMU fusion, EKF)
- UART output format to the flight controller (MAVLink message type and frequency)
- Your latency budget breakdown (capture → detect → localise → transmit)

Evaluation Rubric

Criterion	Strong	Partial	Weak
Detection accuracy (>90% detection rate)	15 pts	8 pts	0
Occlusion handling — no dropped frames	15 pts	7 pts	0
Distance estimation (±0.30 m at 7 m range)	20 pts	10 pts	0
World-frame transform with derivation	20 pts	10 pts	0
Real-time streaming output (<250 ms/frame CPU)	15 pts	7 pts	0
Trajectory plot quality and accuracy	15 pts	7 pts	0
BONUS: Fine-tuning / model justification	+10 pts	+5 pts	0
BONUS: Error analysis vs. ground-truth waypoints	+10 pts	+5 pts	0
BONUS: Kalman filter with state vector explanation	+10 pts	+5 pts	0
BONUS: Jetson edge deployment notes	+10 pts	+5 pts	0

Automatic disqualifiers:

- Hard-coded ground-truth coordinates (output not derived from running the video)
- No coordinate transform derivation (calling cv2.solvePnP without explanation)
- Missing or non-executable run.sh — must run end-to-end from: bash run.sh --video input.mp4 --calib calib.json
- GPU used but not disclosed in README

Required Repository Structure

```
your-repo/
■■■ run.sh                # executable entry point (required)
■■■ track_bin.py          # main Python script
■■■ detector.py           # detection module
■■■ localizer.py          # 3D localisation module
■■■ requirements.txt      # pip dependencies
■■■ README.md             # derivation, GPU info, design decisions, results
■■■ trajectory.png        # required output
■■■ results/
    ■■■ output.csv        # full coordinate log
```

run.sh must install dependencies and run the full pipeline from a single command. Do not commit input.mp4 or calib.json to the repository.

Notes for Candidates

- You will receive a download link for input.mp4 and calib.json separately via email.
- GPU usage is permitted and encouraged. If used, state the exact GPU model, VRAM, and CUDA version in README. Results must be reproducible on equivalent hardware.

• Standard COCO-pretrained models include a trash can / garbage bin class. Note in your README whether you use this directly or fine-tune.

• Questions must be submitted to office@skyscouter.com within the first 6 hours only. After that window, no clarifications will be provided.

• Your submission will be reviewed before a 30-minute technical debrief call where you will walk through your design decisions on a shared screen.

Good luck.